Boku-nano

限られた日本語指示から
Python関数を生成する小型言語モデル



Boku-nano：限られた日本語指示からPython関数を生成するSLM

24種類の整数リスト操作を扱う、小型言語モデルをフルスクラッチ学習

① 制限された日本語指示文

整数リストxsと整数kを受け取り、
偶数だけを残し、
各要素に k を加える
solve 関数を書いてください

決められた操作・文型の範囲

→

② Boku-nano

語彙数2048のトークナイザ
1M, 5M & 15M パラメータ
トークナイザとモデルをゼロから学習
decoder Transformerによる
next token prediction

日本語指示から、python関数を生成

→

③ Python関数

```
def solve(xs, k):
    return [x + k for x in xs
            if x % 2 == 0]
```

参照インタプリタによる実行・検証

作り方と確かめ方

1

中間表現を定義

訓練データ合成のための
中間表現（意味AST）
24基本操作を最大3つ組み合せ

```
{"sequence": [{"filter": ["even"]},
              {"map": ["add_k"]}]}
```

2

合成データ作成

意味ASTから、日本語指示文と
pythonコードを生成

それぞれ基本操作に対して、20通りの
日本語表現とpythonコード合成し、
ルールベースで組み合わせる
訓練データ 約19万件

3

フルスクラッチ学習

トークナイザと
transformer をゼロから学習

15M parameter
8 layers・6 heads
1ヘッドあたり64次元

4

テスト検証・
ベンチマークによる評価

出力されたコードを
参照インタプリタで検証

84,180 / 84,550問 合格

意味ASTは24種類、3操作まで組み合せで12,720通り

基本操作

種類	操作数	例
条件に合う数を残す	10	偶数、奇数、正の値、kより大きい値
一つずつ値を変える	8	kを加える、2倍、絶対値、二乗
順序を変える	3	昇順、降順、現在の順序の反転
一部を取り出す	3	先頭k個、末尾k個、1個おき

操作の組み合わせ

操作数	数え方	意味の種類
1操作	24	24
2操作	24 × 23	552
3操作	24 × 23 × 22	12,144
合計	24 + 552 + 12,144	12,720

意味ASTの例

日本語：奇数だけを残し、各要素を2倍して降順に並べる

```
{
    "sequence": [
        {"filter": ["odd"]},
        {"map": ["mul_const", 2]},
        {"order": "descending"}
    ]
}
```

意味ASTから訓練データを自動生成・検証する

共通の処理の定義：意味AST

```
{"sequence": [
    {"filter": ["even"]}, {"map": ["add_k"]}
]}
```

日本語の指示文

生成の手順

Qwenで各操作の日本語表現を生成

ASTの順に結合し、テンプレートの
`{operations}` をルールベースで埋める。

生成例

整数リストxsと整数kを受け取り、
偶数だけを残し、各要素にkを加える
solve関数を書いてください

検証済みのPythonコード

生成の手順

ASTの操作をPythonコードへ変換

構文を確認し、実行結果を
参照インタプリタの結果と検証

生成例

```
def solve(xs, k):
        return [x + k for x in xs
                        if x % 2 == 0]
```

同じ意味から、日本語とコードを作る

段階	作るもの	例
① 意味を決める	操作と順序の記録	偶数を残す。その後、kを加える
② 指示を作る	日本語の説明	偶数だけを残して、各要素にkを加える
③ 正解を作る	Pythonコード	指示どおりに処理するsolve関数
④ 動作を確認する	入力と出力の比較	参照実装と結果が一致するか

指示と正解を、一つの意味に対応付ける。

一つの意味に、複数の日本語を用意する

共通の意味	指示の言い回し
偶数を残す	偶数だけを残す
偶数を残す	偶数の要素を取り出す
偶数を残す	偶数の値を抽出する

ルールと補助モデルを使い、意味を確認して合成データに採用する。

訓練・検証・テストに意味を分けてから、表現を増やす。

コードの書き方にも変化を付ける

どちらも「偶数だけを残す」正しい書き方。

```
result = [x for x in xs if x % 2 == 0]
```

```
result = []
for x in xs:
    if x % 2 == 0:
        result.append(x)
```

文字列が同じかではなく、動作が同じかを重視する。

学習に使った合成データは192,900件

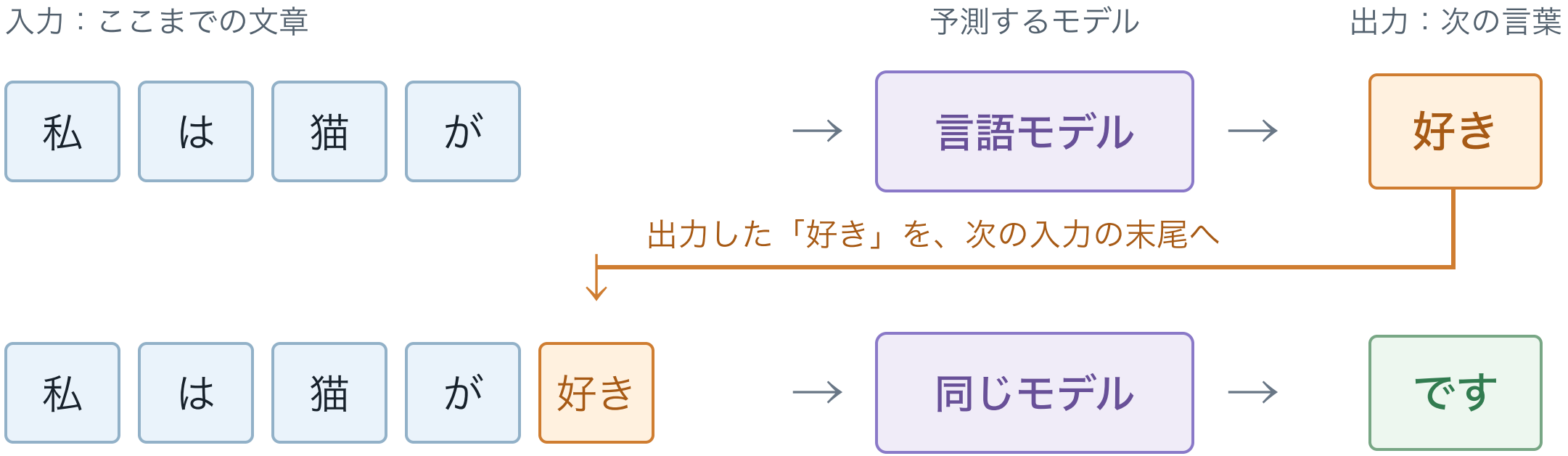
1件は「日本語の指示」と「正解コード」の組。

処理の長さ	訓練データの件数
1操作	460
2操作	8,680
3操作	183,760
合計	192,900

約95%が3操作の合成データ。 単独操作の合成データは少ない。

自己回帰：出力を次の入力に加えて繰り返す

GPTなどの言語モデルは、**直前までの言葉から次を予測**して文章を作る。



> **学習では**、文章の途中までを見せ、実際の次の言葉を当てるように学ぶ。

① ロジットは、確率に変換する前の「候補の点数」

```
# 語彙の各トークンに点数を付ける層を定義
self.lm_head = nn.Linear(
    config.d_model, config.vocab_size,
    bias=False)

# …（初期化の続き・forwardの前半を省略）
# 入力から得た表現を整える
hidden_states = self.final_norm(hidden_states)

# 表現を、全候補の点数（logits）へ変換
logits = self.lm_head(hidden_states).float()

# 推論時は、この点数を返す
if labels is None:
    return CausalLMOutput(
        logits=logits, loss=None)
```

boku_nano.py：330、395–400行から抜粋。
省略を明示。コメントを追加し、改行を調整。

「次に続きそうか」を比べる点数

今の入力に続く候補ごとに、モデルがロジット z_i を計算する。**高い候補ほど選ばれやすい。**

$$z_i = \mathbf{w}_i^\top \mathbf{h}$$

$\mathbf{h}$：ここまでの入力から作った表現

$\mathbf{w}_i$：候補 i に対応する学習済みの重み

ロジットは確率ではない

ロジットは負の値も取れ、合計も1とは限らない。例えば `[2, 1, 0]` は、A・B・Cの点数。

softmaxで確率に変換してから、候補を選ぶ。

② softmaxは、点数を「合計1の確率」に変換する

```
# 入力の最後の位置にある、全候補の点数
# このページでは temperature=1.0 とする
logits = (
    model(input_ids).logits[:, -1, :]
    / temperature
)

# 候補を点数の高い順に並べる
sorted_logits, sorted_indices = torch.sort(
    logits, descending=True, dim=-1
)

# 点数を、合計1の確率分布に変換する
sorted_probabilities = torch.softmax(
    sorted_logits, dim=-1
)
```

evaluate_boku_nano_comparison.py：209–211行。
コメントを追加し、改行を調整。

指数に変換し、全候補の合計で割る

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}$$

z_i：候補 i の点数、p_i：その選択確率

V：候補数、$\sum$：全候補の足し算、$e \approx 2.718$

候補	点数 z_i	e^{z_i}	確率 p_i
A	2	7.389	66.5%
B	1	2.718	24.5%
C	0	1.000	9.0%

$$p_A = \frac{7.389}{7.389 + 2.718 + 1} \approx 0.665$$

点数0でも確率0ではない。 $e^0 = 1$ だから。

③ 選んだトークンを入力に加え、予測を繰り返す

```python
# 日本語指示を含む入力ID列をコピーする
current = prompt_ids.copy()
generated: list[int] = []

# 勾配を計算せず、次のトークンを順に生成
with torch.inference_mode():
    for _ in range(min(
        max_new_tokens,
        model.config.context_length - len(current)
    )):
        values = torch.tensor(
            [current], dtype=torch.long)
        # 最も確率が高いトークンを選ぶ
        token_id = int(torch.argmax(
            model(values).logits[0, -1]).item())
        # 終了トークンなら生成を止める
        if token_id == eos_token_id:
            break
        # 選んだIDを出力と次回の入力へ追加
        generated.append(token_id)
        current.append(token_id)
```

Python推論から抜粋。コメントを追加し、改行を調整。
export_boku_nano_onnx.py：232–241行

`logits[0, -1]` は先頭の入力系列の **最後の位置** にある、全候補の点数。

自己回帰の中心は `current.append`

1. **ここまでの入力**から次の候補を予測する。
2. 候補を1つ選び、**入力末尾に追加**する。
3. 伸びた入力を使って、**同じモデル**でもう一度予測する。

各ステップで、最も確率が高いトークンを選ぶ

選んだトークンを入力に加え、次の予測でも **最も確率が高いトークン** を選ぶ。終了トークンか長さの上限まで繰り返す。

softmaxで確率に変換しても、候補の順位は変わらない。

そのため、コードでは **ロジットが最大の位置を求める** `argmax` で、同じトークンを選べる。

④ 温度は、softmaxに入れる点数の差を変える

```
# 全候補のロジットを温度Tで割る
logits = (
    model(input_ids).logits[:, -1, :]
    / temperature
)

# 候補のIDを保持して点数順に並べる
sorted_logits, sorted_indices = torch.sort(
    logits, descending=True, dim=-1
)

# 調整後の点数をsoftmaxで確率にする
sorted_probabilities = torch.softmax(
    sorted_logits, dim=-1
)
```

evaluate_boku_nano_comparison.py：209–211行。
確率に従って抽選する場合の実装から抜粋。
今回のBoku評価では、最も確率が高いトークンを選ぶ。

抽選する場合の分布を温度で調整する

$$p_i(T) = \frac{e^{z_i/T}}{\sum_{j=1}^{V} e^{z_j/T}}, \quad T > 0$$

T：温度。$T = 1$ なら元のsoftmaxと同じ。

AとBの点数が2と1なら、差は1。割った後の差は $1/T$ になる。

温度 T	Aの点数 $2/T$	Bの点数 $1/T$	差
0.5	4	2	2
1	2	1	1
2	1	0.5	0.5

小さい温度 → 差が広がる → 高い点数に集中

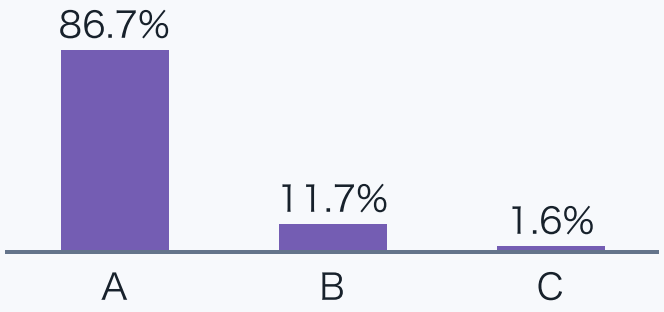
大きい温度 → 差が縮まる → 他の候補も選ばれる

⑤ 同じロジットでも、温度で「選ばれる割合」が変わる

元の点数はずっと A=2、B=1、C=0。温度だけを変えた確率分布を比べる。

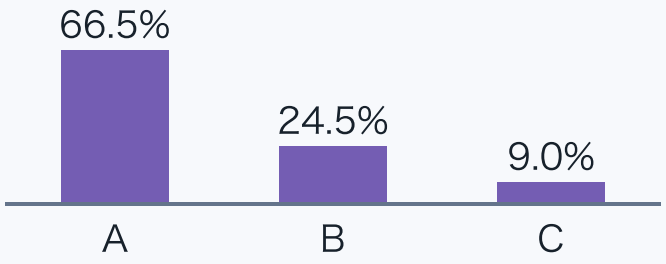
T = 0.5：Aに集中

割った後の点数：[4, 2, 0]



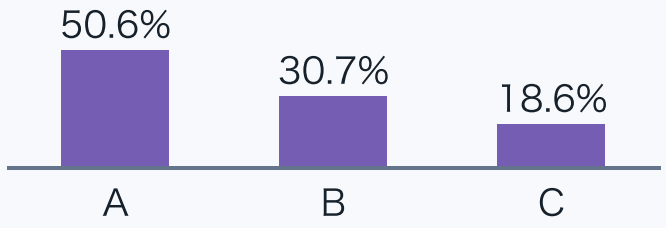
T = 1：基準

割った後の点数：[2, 1, 0]



T = 2：B・Cも選びやすい

割った後の点数：[1, 0.5, 0]



AはBの何倍選ばれやすいか（棒の高さは共通尺度：0〜100%）

$$\frac{p_A(T)}{p_B(T)} = e^{(z_A - z_B)/T} = e^{1/T}$$

$T = 0.5$ なら約 7.39倍、$T = 1$ なら約 2.72倍、$T = 2$ なら約 1.65倍。

正の温度では順位は同じ。抽選のばらつきが変わる。温度を上げても正解率が上がるとは限らない。

$T \to 0^+$ では最大候補に集中（最大が1つの場合）。$T = 0$ の除算はせず、最大の候補を直接選ぶ。

五つのテストで、生成コードを実行する

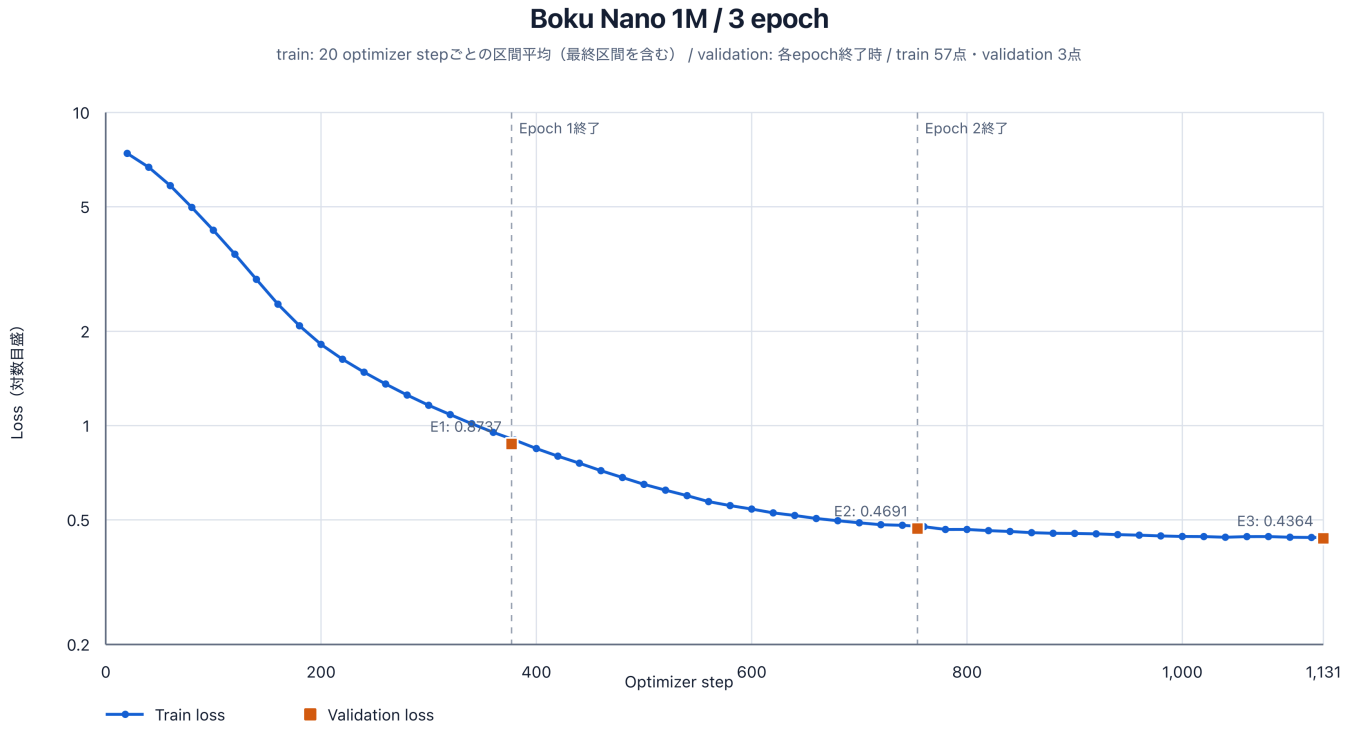
テスト	確かめること	指示の件数
通常	未学習の操作の並びを扱えるか	24,040
組合せ	訓練から隔離した操作対を組めるか	13,400
日本語の言い換え	未学習の30表現を理解できるか	30
同じ操作の反復	同じ操作を2回・3回実行できるか	23,040
境界値	空リストなどでも正しく動くか	24,040

1指示につきコードを1候補生成し、全検査を通ったら合格。

通常と境界値では同じ指示を使い、検査する入力を変える。

1M：3 epochまでの損失の推移

3 epochの学習。訓練損失は20 stepごと、検証損失は各epoch終了時の実測値。

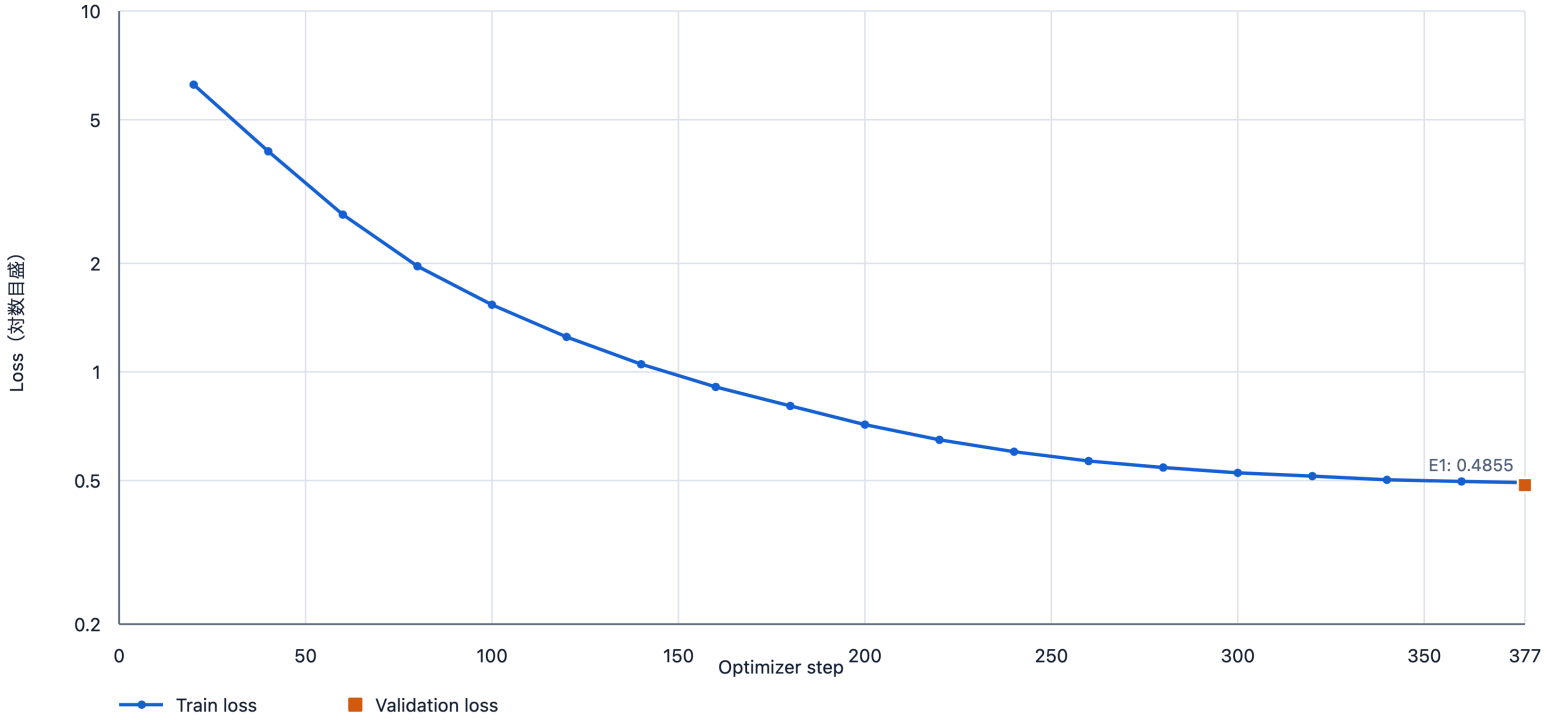


val loss：1 epoch 0.8737 ／ 2 epoch 0.4691 ／ 3 epoch 0.4364

20 stepごとのvalidationは未記録。図は保存済みの3点を表示。

5M：損失の推移

1 epochの学習。訓練損失と、検証データで測った損失を示す。

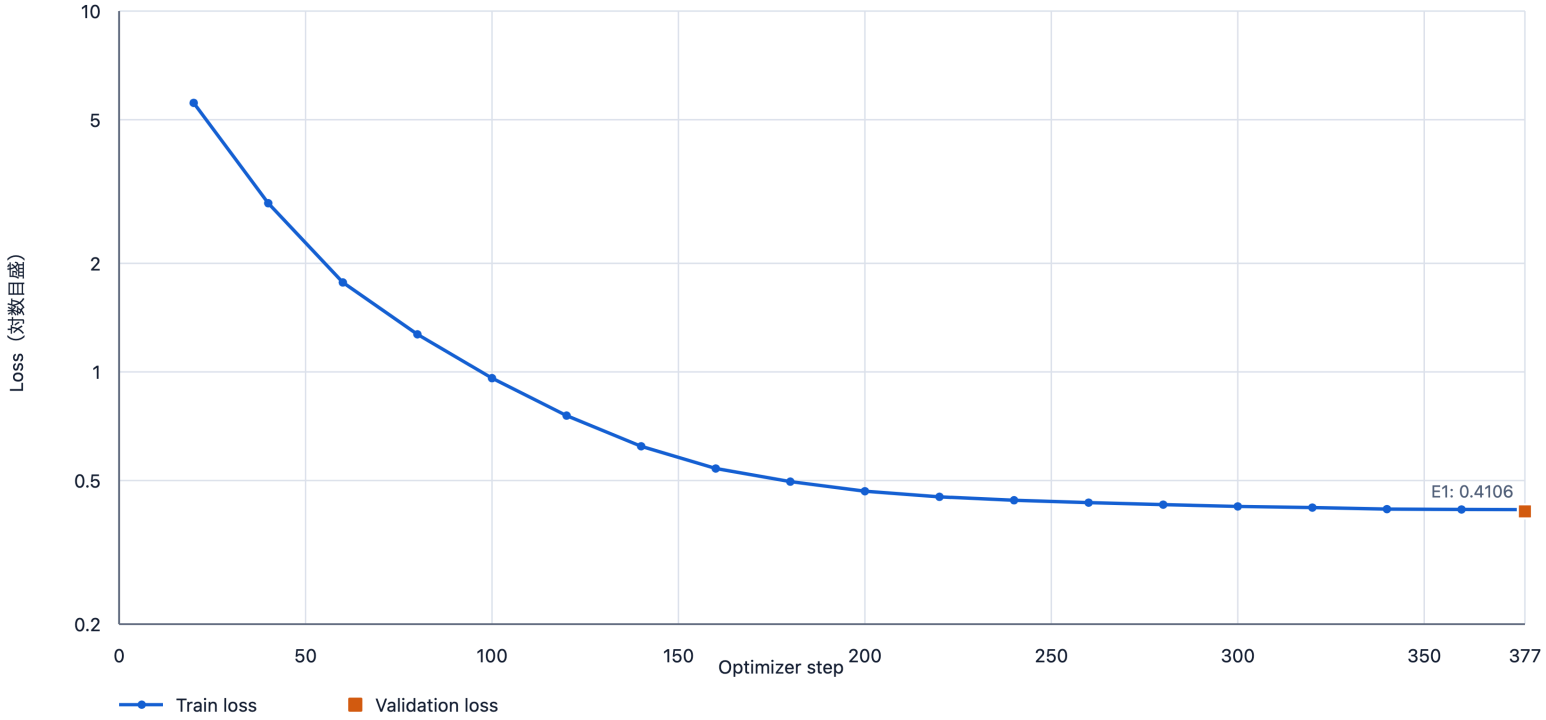


学習終了時の val loss：0.4855

val lossは終了時の1点。途中の検証値は保存されていない。

15M：損失の推移

1 epochの学習。訓練損失と、検証データで測った損失を示す。



学習終了時の val loss：0.4106

val lossは終了時の1点。途中の検証値は保存されていない。

1 epochの結果：モデル規模による違い

五つのテストを合計した参考値。分母は84,550件。

モデル	合格数	合格率
1M	1,467件	1.74%
5M	81,557件	96.46%
15M	84,180件	99.56%

今回の条件では、5Mと15Mは高い全体合格率を示した。

ただし、件数の少ない言い換えテストは全体値に表れにくい。

全体が高精度でも、言い換えには弱い

15Mモデル・1 epochのテスト別結果。

テスト	合格数	合格率
通常	24,003 / 24,040	99.85%
組合せ	13,188 / 13,400	98.42%
日本語の言い換え	**15 / 30**	**50.00%**
同じ操作の反復	22,971 / 23,040	99.70%
境界値	24,003 / 24,040	99.85%

全体の99.56%だけでは、日本語表現への弱さが見えない。

先行研究：日本語の書き方で正解率が変わる

Gan and Mori（2023）は、意味や指示が近い5種類のプロンプトを比較した。

プロンプトは、モデルに渡す指示文や質問文のこと。

日本語の課題	モデル	正解率の範囲
2文の意味関係を分類するJNLI	GPT-4	25.44〜49.21%

意味が近くても、書き方によって性能が大きく変わる場合がある。

出典：Gan and Mori, PACLIC 2023, Table 3

先行研究：言い換えで回答の傾向も変わる

Takayama et al.（2026）は、日本語を含む「はい／いいえ」の質問を評価した。

表現の変え方	日本語での観察
同意を求める「〜ですよね？」	道徳判断で「はい」が増える傾向
反対語への置き換え	答えの反転を考慮しても、一貫性が低下

同じ判断を求めても、表現によって答えが動く。

モデルの評価には、複数の言い回しを含める必要がある。

出典：Takayama et al., LREC 2026, Table 1

今回は、指示の冒頭だけを変えて比較する

条件	モデルに渡す指示
変更前	整数リストxsから偶数だけを残すsolve関数を書いてください。
変更後	整数リストxsに対して、偶数だけを残すsolve関数を書いてください。

後半の操作の指示をそろえ、冒頭だけが異なる**14操作**を比較。

同じモデルを使い、追加学習せず、同じ入力でコードを検査する。

5Mモデルでは、12件の合格が1件に減った

5M・1 epoch。比較した14操作の合格数。

指示の冒頭	合格数を表す図	合格数・合格率
整数リストxsから	■■■■■■■■■■■■□□	12 / 14・85.7%
整数リストxsに対して、	■□□□□□□□□□□□□□	1 / 14・7.1%

■は合格1件、□は不合格1件。左から合格数を並べた図。

冒頭を変えただけで、ほとんど正しく生成できなくなった。

失敗例：偶数を残す指示を別の処理にした

変更後の指示：**整数リストxsに対して、偶数だけを残すsolve関数を書いてください。**

	指示どおりの処理	変更後に生成した処理
操作	偶数だけを残す	順序を反転し、k以上の値を残す
検査入力	`xs = [0]`、`k = 1`	同じ入力
出力	`[0]`	`[]`

構文は正しかったが、**処理の意味が変わった。**

表現への強さは、モデルによって違う

すべて1 epoch。冒頭だけを変えた14操作の合格数。

モデル	「xsから」	「xsに対して、」
5M	12 / 14	1 / 14
15M	12 / 14	12 / 14

15Mでは合格数を保った。 すべてが同様に崩れるわけではない。

学習データの文型との結び付きが考えられる

訓練データで「整数リストxsに対して、」と始まる指示は84件。

その文型で学習した処理	件数
1操作	0
2操作	1
3操作	83

この文型を、複数の操作と結び付けて覚えた可能性がある。

原因の確定には、合成データを変えた追加比較が必要。

モデルが情報を読み取る仕組み

アテンションは、次のコードを作るときに、どこから情報を読むかを決める仕組み。

参照できる情報	内容
日本語の指示	「偶数」「残す」など、求める処理
すでに生成したコード	変数名、構文、処理の続き
開始の印など	入力の始まりや区切り

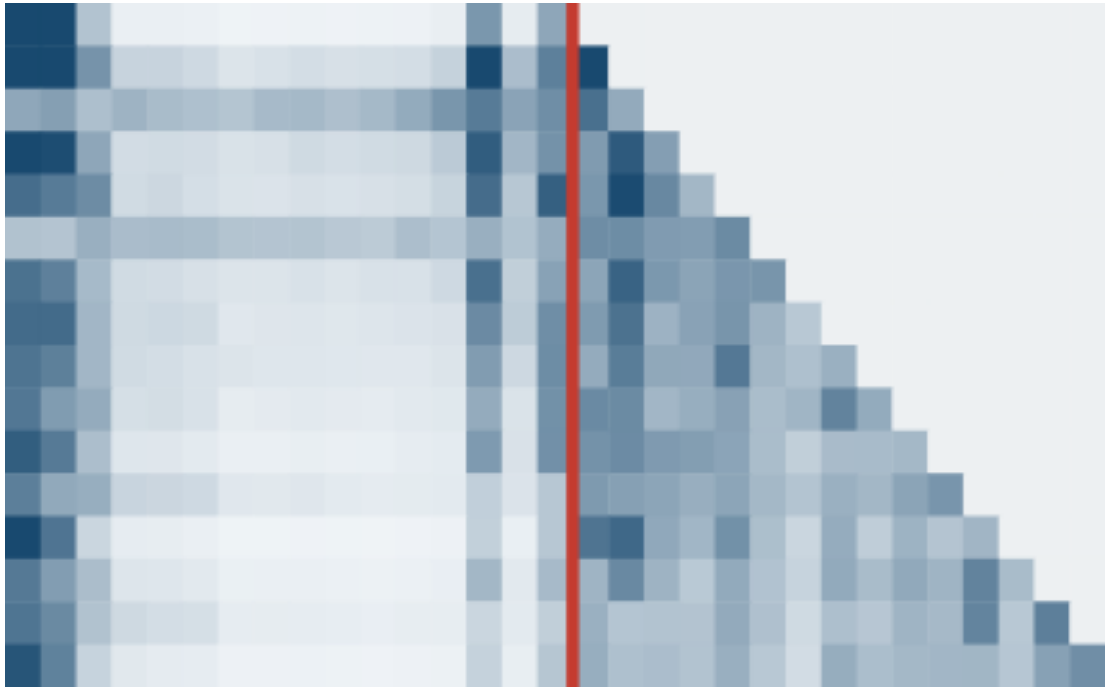
参照割合だけでは、その情報が必要だったかは確定できない。

情報の受け渡しを止めて、生成結果が変わるかを調べる。

アテンションの図の読み方

入力の指示・区切り　　生成済みコード

生成
ステップ
↓



薄い：弱い

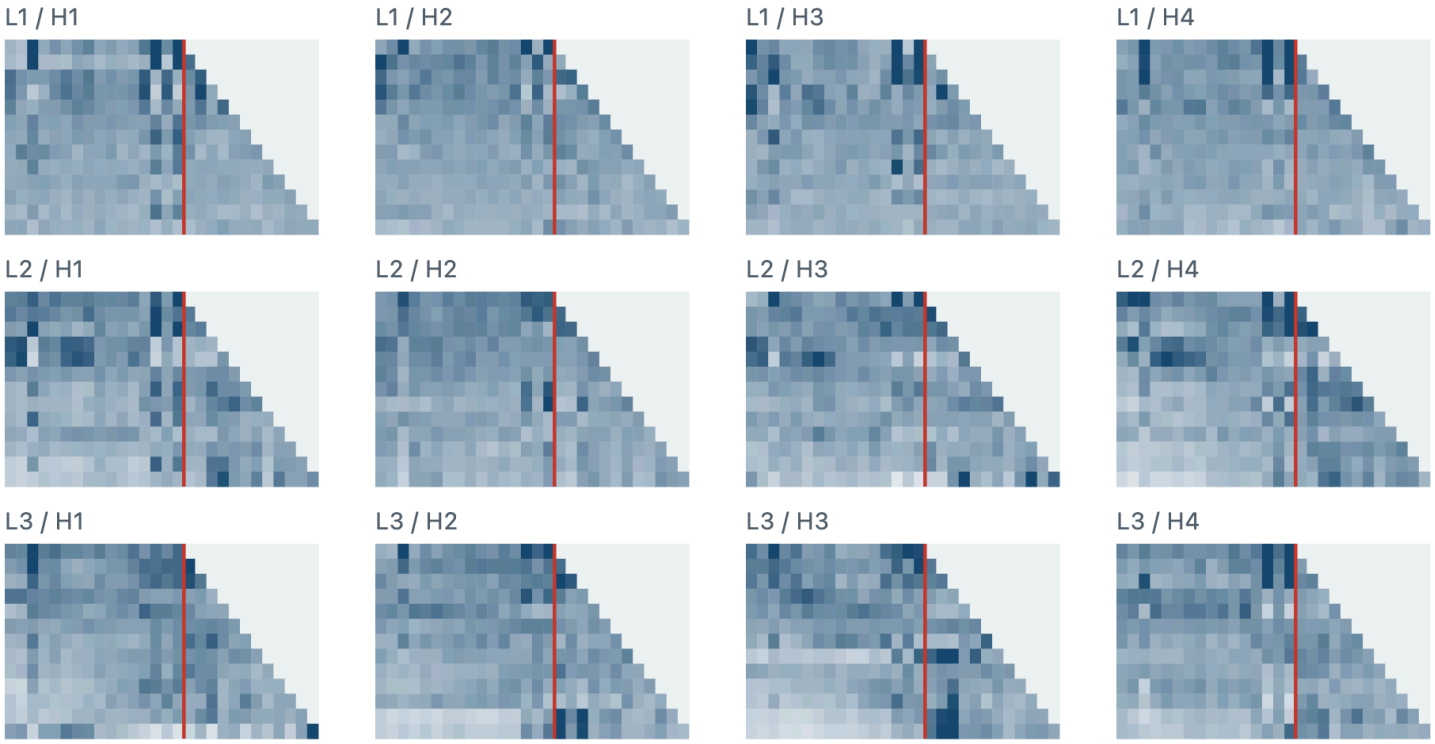
濃い：強い

横：参照するトークン位置 →

濃い位置ほど強く参照。15M・1エポックのL8・H5の例

1M：attention の可視化

同じ指示：**各要素を絶対値にして、降順に並べる**

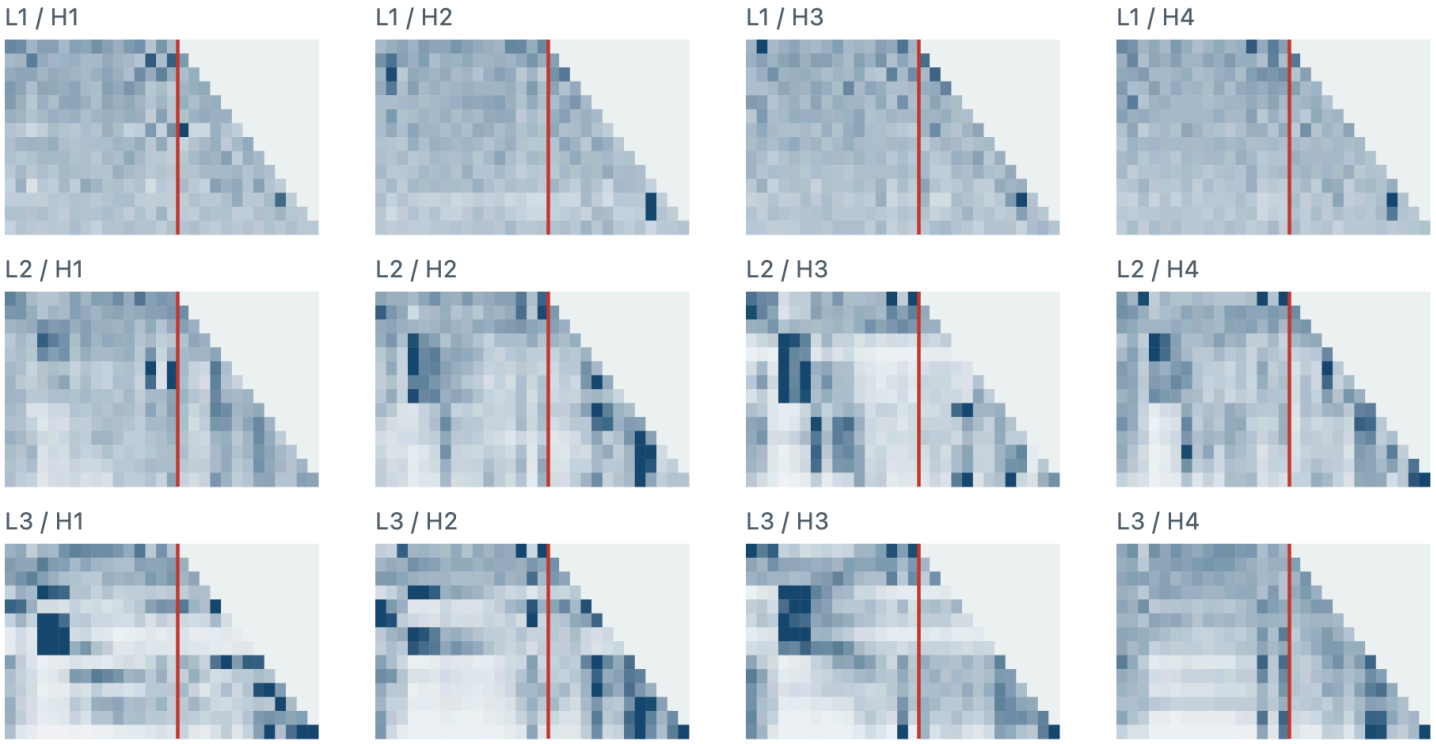


生成結果：未定義の result を参照し、この指示では実行できなかった。

色は各モデル内の相対的な強さ。モデル間で濃さを直接比較しない。

5M：attention の可視化（L1〜L3）

同じ指示：**各要素を絶対値にして、降順に並べる**

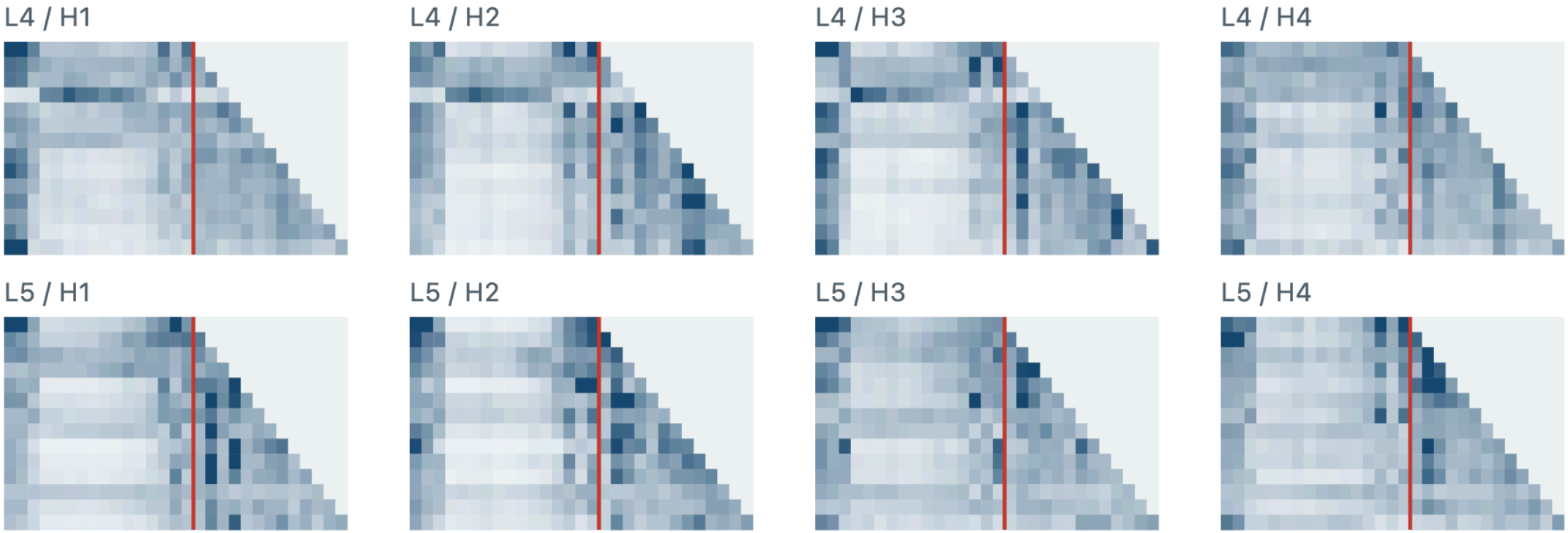


生成結果：絶対値の処理は生成したが、k加算が入り、降順の処理が抜けた。

色は各モデル内の相対的な強さ。モデル間で濃さを直接比較しない。

5M：attention の可視化（L4〜L5）

同じ指示：**各要素を絶対値にして、降順に並べる**

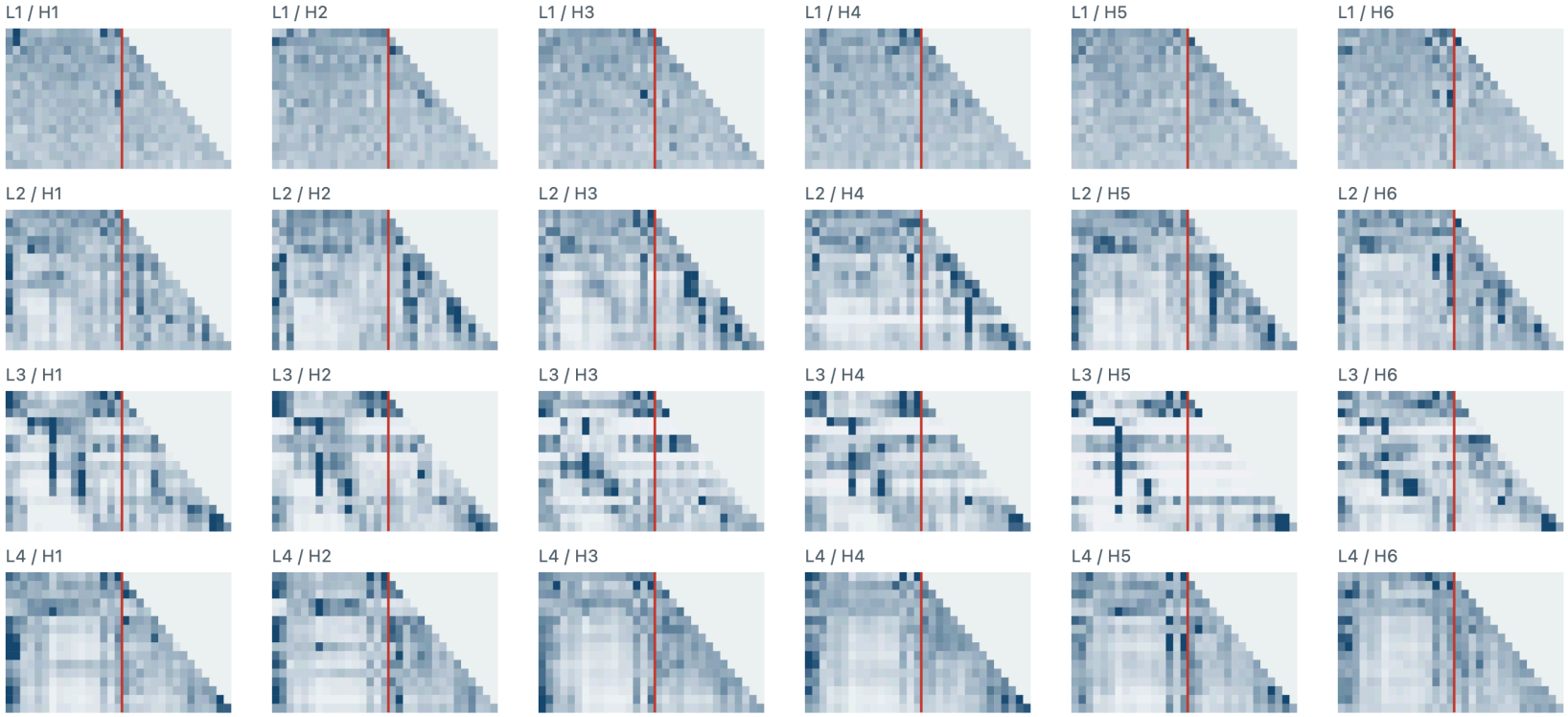


生成結果：絶対値の処理は生成したが、k加算が入り、降順の処理が抜けた。

色は各モデル内の相対的な強さ。モデル間で濃さを直接比較しない。

15M：attention の可視化（L1〜L4）

同じ指示：**各要素を絶対値にして、降順に並べる**

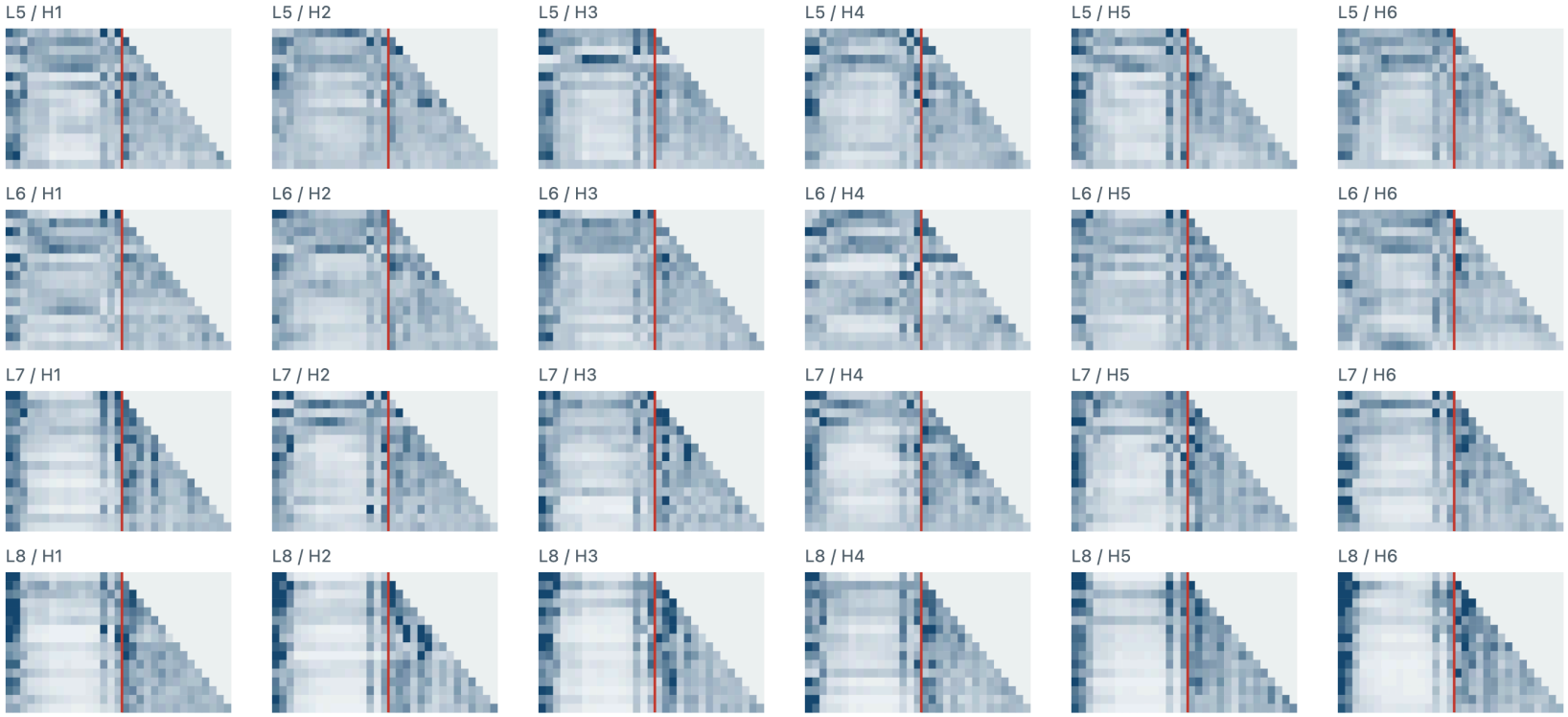


生成結果：絶対値を取って降順に並べるコードを生成した。

色は各モデル内の相対的な強さ。モデル間で濃さを直接比較しない。

15M：attention の可視化（L5〜L8）

同じ指示：**各要素を絶対値にして、降順に並べる**



生成結果：絶対値を取って降順に並べるコードを生成した。

色は各モデル内の相対的な強さ。モデル間で濃さを直接比較しない。

指示の情報を止めると合格数が落ちる

1 epochモデル。学習時に準拠した文型で、24操作を診断。

モデル	通常の生成	指示の情報を止めた生成
1M	3 / 24	0 / 24
5M	22 / 24	0 / 24
15M	22 / 24	2 / 24

指示から受け取る情報が、操作の選択に寄与している。

24操作の内部診断。冒頭変更の14操作、固定5テストとは別。

評価では、表現を変えた場合も確かめる

調べること	今回、分かったこと
決めた文型での動作	5M・15Mは高い全体合格率を示した
未学習の言い換え	15Mでも30件中15件の合格
指示の冒頭の変更	5Mは14件中12件から1件へ低下

一種類のプロンプトでの精度に加え、表現を変えても性能が保たれるかを評価する必要がある。

組み合わせ汎化ベンチマークの前提

各操作を表す日本語表現は、Bokuの訓練データに登場する。
ただし、完成した指示文は訓練データと完全一致しない。

評価872問と訓練192,900件を照合し、指示全文の一致は0件。

課題	訓練と評価で何が変わるか
複数操作で学んだ表現を1操作で使う	複数操作の指示でだけ使った言い方を、単独操作に使う
初めて組み合わせる2操作	訓練では同じ指示に登場しない2操作を組み合わせる
3操作で学んだ順序を2操作で使う	3操作中の既知の順序を、未学習の2操作列として使う

未知の日本語表現への強さとは分けて、**既知の表現を異なる構成で使う能力**を調べる。

データセットの構成：9種類・872問

操作数	問題の種類	問数	構成
1	複数操作で学んだ表現を1操作で使う	96	24操作 × 4表現
2	初めて組み合わせる2操作	100	5ペア × 両順序 × 10表現
2	同じ操作を2回	96	24操作 × 4表現
2	3操作で学んだ順序を2操作で使う	88	88種類の操作列
3	初めてのペアを含む3操作	100	100種類の操作列
3	同じ操作を3回	96	24操作 × 4表現
4	同じ操作を4回	96	24操作 × 4表現
4	異なる4操作を組み合わせる	100	100種類の操作列
4	2操作の組を2回繰り返す	100	A・B・A・Bの100種類

各問題でコードを1つ生成。**179通りの共通テスト入力ですべて合格した問題の割合**を正答率とする。

Astraの評価設定：思考low・ツール使用を禁止

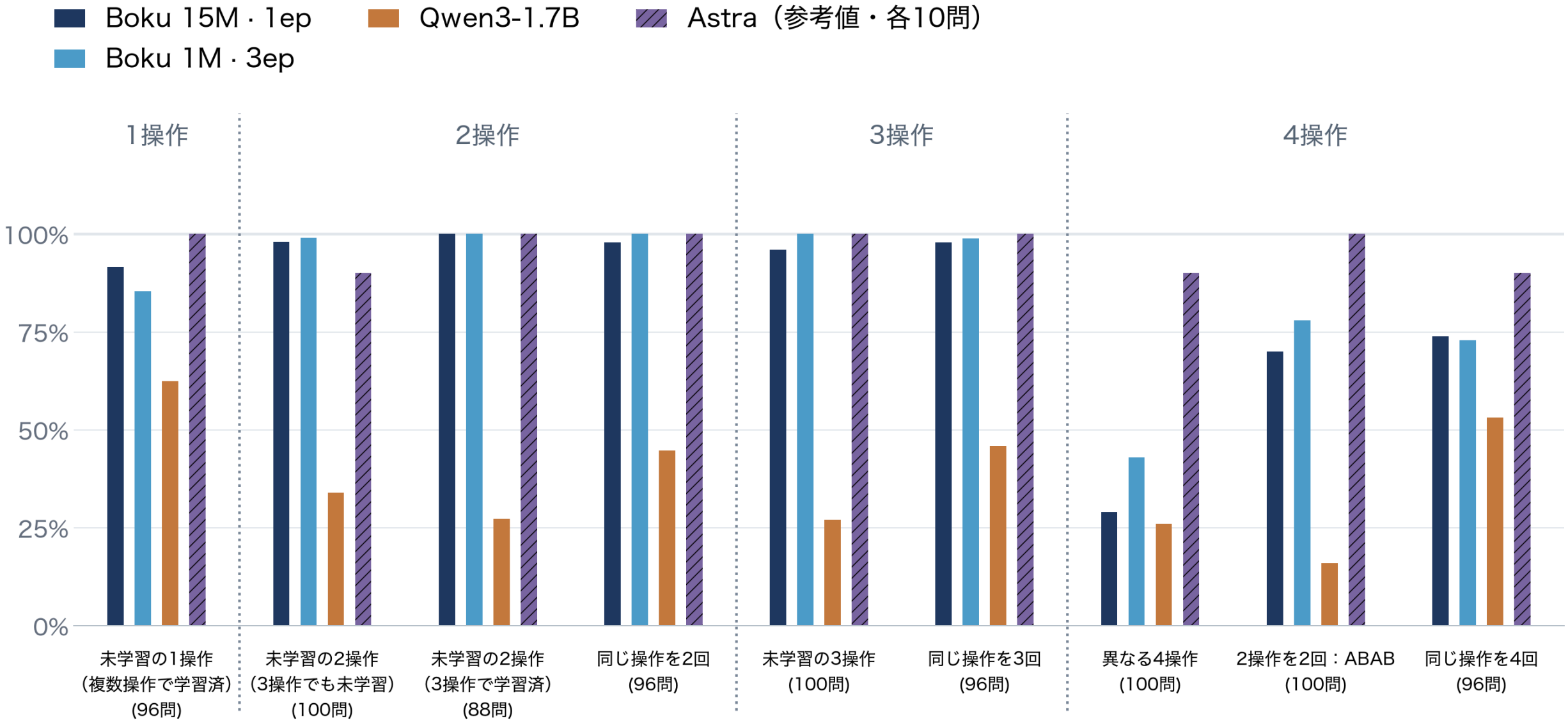
項目	設定
モデル・推論設定	`gpt-6-astra` ／ reasoning effort：**low**
評価対象	872問から固定seedで抽出した各カテゴリ10問、計90問
会話・回答	1問ごとに独立チャット。最初の回答1回を採点。修正・再生成なし
ツール	使用禁止を指示。**90問すべての実行記録で使用0件を確認**

各問題の冒頭に与えた指示：

> ツールは一切使用せず、このメッセージだけから回答してください。ファイル閲覧、検索、コード実行、他のエージェントへの依頼は行わないでください。回答は一度だけ、指定されたコードのみを出力してください。

ツールの技術的な無効化ではなく、**指示による禁止と記録確認**。採点は回答保存後に行い、結果をAstraへ返していない。

問題カテゴリ別のモデル比較



本評価：カテゴリ欄の問題数／Astra参考値：各10問・計90問（独立チャット・low・ツール使用なし）。評価集合・実行条件が異なるため参考比較。

高得点の課題と、残る課題を分けて読む

問題の種類	Boku 15M・1ep	Boku 1M・3ep	Qwen3-1.7B	Astra・参考値
初めて組み合わせる2操作	98 / 100	99 / 100	34 / 100	9 / 10
初めてのペアを含む3操作	96 / 100	100 / 100	27 / 100	10 / 10
異なる4操作を組み合わせる	29 / 100	43 / 100	26 / 100	9 / 10

Astraだけ各カテゴリ10問。**分母と対象問題が異なるため、参考比較。**

- **今回の2・3操作の課題では、小規模モデルにも有望な結果がある。**
- Bokuの4操作では誤りが多く残る。どの操作を落とすか、順序を変えるかを分析する。
- 1Mは3epoch、15Mは1epoch。生成条件も異なり、規模だけの優劣とは解釈しない。

Astraとの比較：同じ90問に揃えた結果

Boku・Qwenも、Astraに出した**同じ各10問**の結果だけを再集計。

問題の種類	Boku 15M・1ep	Boku 1M・3ep	Qwen3-1.7B	Astra・low
複数操作の表現を1操作に使用	10 / 10	8 / 10	6 / 10	10 / 10
初めて組み合わせる2操作	10 / 10	10 / 10	4 / 10	9 / 10
3操作の順序を2操作に使用	10 / 10	10 / 10	2 / 10	10 / 10
同じ操作を2回	10 / 10	10 / 10	5 / 10	10 / 10
初めてのペアを含む3操作	10 / 10	10 / 10	0 / 10	10 / 10
同じ操作を3回	10 / 10	10 / 10	5 / 10	10 / 10
異なる4操作	3 / 10	6 / 10	3 / 10	9 / 10
2操作の組を2回（ABAB）	4 / 10	6 / 10	0 / 10	10 / 10
同じ操作を4回	10 / 10	8 / 10	6 / 10	9 / 10
全90問	**77 / 90**	**78 / 90**	**31 / 90**	**87 / 90**
正答率	**85.6%**	**86.7%**	**34.4%**	**96.7%**

Astraは87/90正解。各カテゴリは**1問差で10ポイント**。問題は同じでも、推論・実行条件は完全には揃っていない。

今回の結果の意義と、次の研究課題

今回のベンチマークでは、小規模モデルの組み合わせ汎化について有望な結果が得られた。

今後は比較条件を揃え、より大きなモデルとの比較を追加するとともに、**評価設計と作成手順を明文化する。**

- 比較条件：訓練回数、プロンプト、生成上限などを記録し、揃えられる条件を揃える。
- 評価設計：何を既知・未知とするか、どの誤りを検出したいかを定める。
- 作成手順：問題の選定、訓練との照合、採点方法、再現手順を公開可能な形に整理する。

ベンチマークの役割

モデルの能力と限界を観測し、次に取り組む研究課題を明確にする。

- 入出力の成功例だけでは、能力を獲得したか判断できない。共通の基準で定量評価する。
- 正誤を検証できる課題では、評価基準を明確にしやすい。小規模モデルでも改善を確かめられる。
- 得点だけでなく、どの条件で成功し、どの条件で失敗するかを見る。

今回なら、**2・3操作での高得点と、4操作で残る誤り**が、次の分析対象を教えてくれる。

自分の研究でベンチマークを作るには

1. **測りたい能力を先に決める。** 例：既知の操作を、新しい組み合わせで実行できるか。
2. **目的に沿って課題と正解基準を作る。** 何を変え、何を固定するかを決める。
3. **共通条件で測り、成功と失敗を分析する。** 作成手順と比較条件も記録する。
4. **高得点で差が見えなくなったら、評価の役割を見直す。** 元の結果を残し、次に測る能力を定める。

得点を上げる研究と、能力向上を観測できる評価を作る研究を、両方考える。

この結果が示す範囲

- 訓練は24操作を最大3つ組み合わせるコード生成。追加評価では4操作まで調べた。
- 冒頭変更の比較は14操作。日本語能力全体の測定ではない。
- 学習は一つの乱数設定。繰り返し実験のばらつきは未確認。

今後は文型の偏りを減らした合成データで、表現への強さを再評価する。

まとめ

限定したコード生成は、小さなモデルでも1 epochで学習できた。

表現を変えた場合の性能と、操作を組み合わせる能力は、分けて確かめる必要がある。

学習済みモデルをブラウザで試す

ONNX形式で保存したモデルを、ONNX Runtimeでブラウザ内実行。Python環境・推論サーバーは不要。

デモの流れ	役割
① 自由な日本語を入力	作りたい処理を伝える
② Qwen3-0.6Bで文型を整える	自由文を決められた文型（CNL）に変換。コードは書かない
③ JavaScriptで指示を検査	24操作・最大3操作・定型文法に合うか確認
④ Boku-nanoでコードを生成	選択した学習済みモデルがPython関数を出力

ブラウザデモを開く

Boku1-nano 全体図

日本語の限定的な指示から solve(xs, k) を生成する小型Decoder-only Transformer。意味ASTを起点に、データ生成・学習・評価をすべて同じ意味仕様でつなぐ。

1 意味

24の基本操作
抽出 10：偶数・奇数・>k・≥k・<k・≤k・kの倍数・正・負・ゼロ
変換 8：+k・−k・×k・2倍・3倍・符号反転・絶対値・二乗
並べ替え 3／切り出し 3

→

意味AST　12,720仕様
24操作から1〜3操作を順序付きで選ぶ（重複なし）
24 ＋ 24×23 ＋ 24×23×22 ＝ 12,720
`{"sequence": [{"filter": ["even"]}, {"map": ["add_k"]}]}`
※日本語・コード・期待値・splitの共通の根拠

→

参照インタプリタ（期待値の計算）
意味ASTを入力リストに適用して、正しい出力を計算する
operator・itertools・heapq・deque・while で実装し、生成されるsolve関数の表現空間の外側に置く
→ 正解コードの検証（2）とhidden test採点（4）に使う
※参照実装と生成コードが同じ式を書いて誤りを共有しないため

↓ 意味ASTを分割　　↓ 期待値で検証（参照インタプリタ → 正解コード生成）

2 データ

意味ASTの分割
12,720 から 670 を組合せ汎化へ
残り 12,050 をハッシュ順で分割
　train　9,646（1操作24は全部）
　validation　1,202
　通常テスト　1,202（境界値も流用）
別枠：同一操作の反復 1,152
　　日本語言い換え 30指示

→

正解コード生成（自作テンプレート）
style_specでfor／内包表記・変数名・スライス表記などを変える
277,440候補 → 構文・Safe-AST・signature・入力非破壊・
41入力で参照インタプリタと出力一致、を全部通したものだけ

日本語指示生成
ルール：承認済み表現辞書（train 522件）を定型文へ連結
教師言い換え：Qwen3-4B-AWQ が意味と順序を保って言い換え
※Qwenは日本語だけを作り、コードや重みは提供しない

→

評価JSONL　108,590件
validation 24,040
正式5テスト 84,550問
hidden入力はコード作成時と別

訓練JSONL　192,900件
1操作 460／2操作 8,680／3操作 183,760
ルール文 96,510＋言い換え 96,390
1意味ASTあたり原則20件

⋯⋯

1レコードの中身
instruction_ja（日本語指示）
semantic_ast（意味AST）
reference_code（正解コード）
style_spec・code_style
split・tests・verification
SHA-256・seed・生成器version
教師モデル由来かどうかの来歴

↓ train の指示とコード（訓練JSONLから）

3 学習

BPEトークナイザ
ByteLevel BPE・語彙 2,048
trainの日本語指示と正解コードだけから学習
（評価データは入れない）
min_frequency 5・最大token長 24
unknown token 0件

→

学習系列とloss
`<|bos|><|task|> {instruction_ja}`
`<|code|> {reference_code}<|eos|>`
lossはコードとEOSだけ（指示は条件として読むだけ）
batch 512件・1エポック 377 step・AdamW・bf16

→

Decoder-only Transformer（ランダム初期値から学習）
RoPE・RMSNorm・SwiGLU・通常のMulti-Head Attention・最大系列長 256
　1M　d=128・3層・4 head
　5M　d=256・5層・4 head
　15M　d=384・8層・6 head
3つの規模を、それぞれ1エポック学習（RTX 5090）

↓ 学習済みモデル　　↓

4 評価

hidden test評価（正式5テスト 84,550問）
greedy生成 → 構文 → Safe-AST → signature → 隔離実行
→ hidden入力すべてで参照インタプリタと同じ出力なら合格
文字列一致ではなく、実行結果で判定する
　通常 24,040　　組合せ汎化 13,400　　日本語言い換え 30
　同一操作の反復 23,040　　境界値 24,040
※attention診断の24問（1操作）はこれとは別の解析用テスト

→

主な結果（1エポック・正式5テスト合計）
15M　99.56%（84,180 / 84,550）
5M　96.46%（81,557 / 84,550）
1M　1.74%（1,467 / 84,550）
※弱点：未学習の言い換え（15Mでも30問中15問）

内部解析
Q・K・Vを取り出しattentionを計算
head別のattention配分・Value norm・
BOS sink・attention rollout
head／位置ablation：NLL差（nat）と
再生成コードの合格数で因果を確認

5 デモ

自由な日本語
例：全部プラスにして大きい順に並べて

→

Qwen3-0.6B（ブラウザ）
自由文をCNLへ翻訳するだけ
追加学習なし・コードは書かない

→

JavaScript検査
24操作・最大3操作・
定型文法に合うか確認

→

Boku1-nano ONNX
学習済みモデルを選んで
ブラウザ内でgreedy生成

→

Pythonコード
`def solve(xs, k) -> list[int]`

自作ルール・決定的な処理　Qwenを使う処理（日本語のみ）　データ・出力　学習・モデル　評価・解析　↓ 参照インタプリタによる検証